

Markov Chain Monte Carlo

MCMC: 독립 표본 대신 Markov chain 으로 표본을 만든다

Markov chain · Metropolis–Hastings · Random Walk Chain · Gibbs Sampling · 수렴 진단

교재 : Givens & Hoeting, Computational Statistics, 2nd ed., Ch. 1.7 & Ch. 7

목차

01

Markov chain 기초

전이확률, 정상분포, ergodic theorem, detailed balance

02

Metropolis-Hastings

알고리즘, 왜 맞는가, independence chain

03

Random walk chain

Metropolis 알고리즘, proposal scale 조정

04

Gibbs sampling

full conditional, 이변량 정규, 정규모형 사후분포

05

수렴 진단과 구현

burn-in, ACF, ESS, Gelman-Rubin $\hat{R}$

+

정리

방법 비교와 참고문헌

01

Markov chain 기초

MCMC의 원리 : 정상분포가 목표분포 f 인 chain을 만들고 오래 돌린다

Markov chain 과 전이확률

$$P(X^{(t+1)} = j \mid X^{(0)}, \dots, X^{(t)} = i) = P(X^{(t+1)} = j \mid X^{(t)} = i) = p_{ij}$$

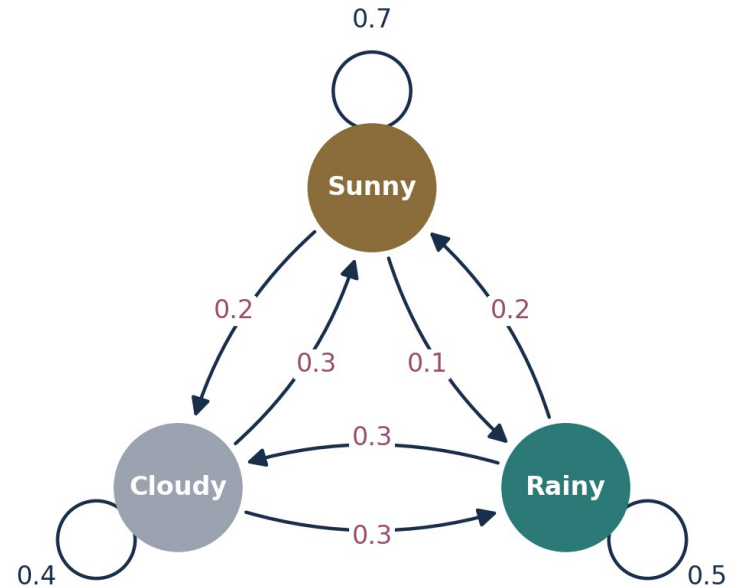
" 미래는 현재에만 의존하고 과거와는 무관 " (Markov property)

예제 : 날씨 chain (상태 1 = Sunny, 2 = Cloudy, 3 = Rainy)

$$P = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{pmatrix}, \quad \sum_j p_{ij} = 1$$

$$\pi^{(t)} = \pi^{(0)} P^t, \quad P(X^{(t)} = j \mid X^{(0)} = i) = (P^t)_{ij}$$

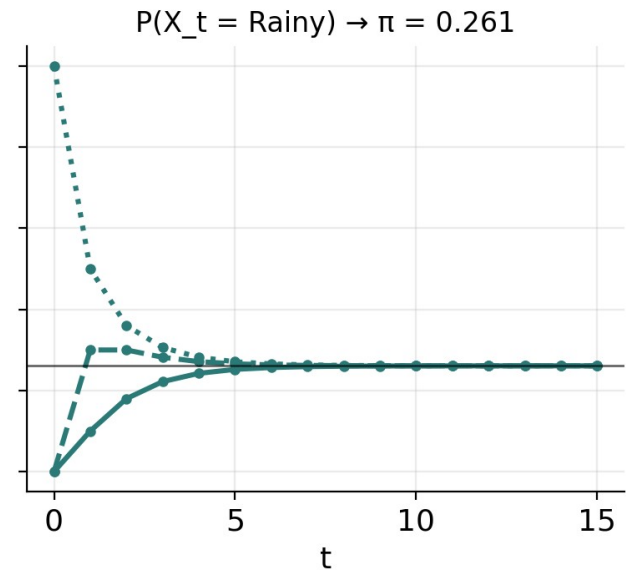
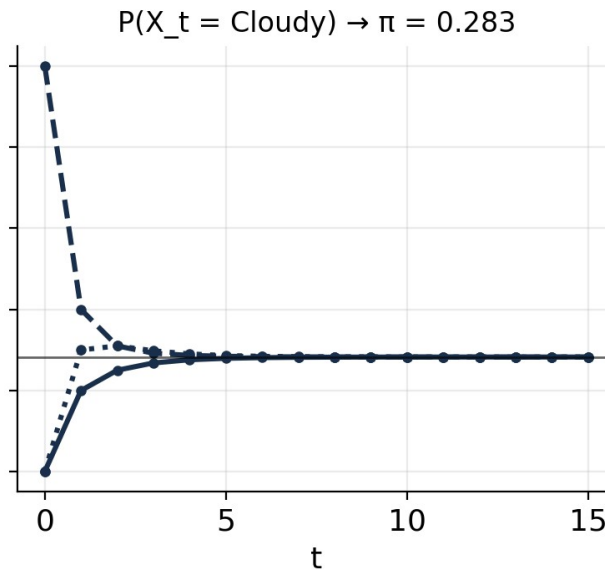
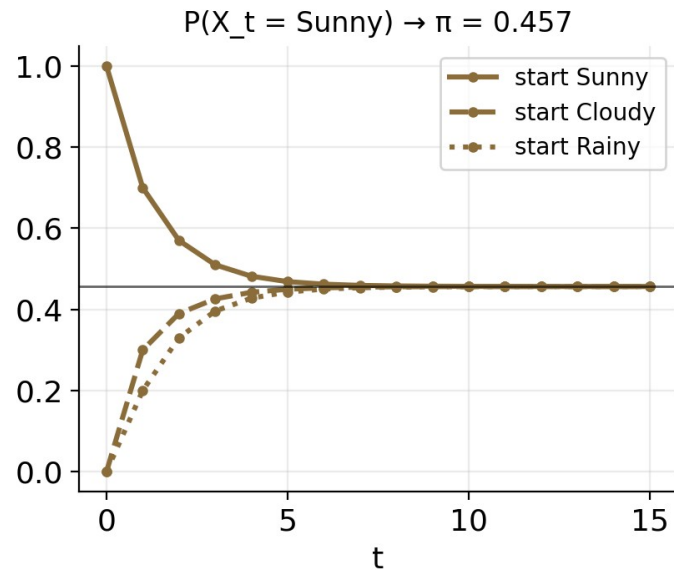
P의 i행 = 오늘 상태가 i일 때 내일의 분포. t 단계 뒤의 분포는 P를 t번 곱한다.



정상분포 (stationary distribution)

$$\pi = \pi P, \quad \sum_i \pi_i = 1$$

P 를 한 번 더 곱해도 변하지 않는 분포 = P 의 고유값 1 에 대한 왼쪽 고유벡터

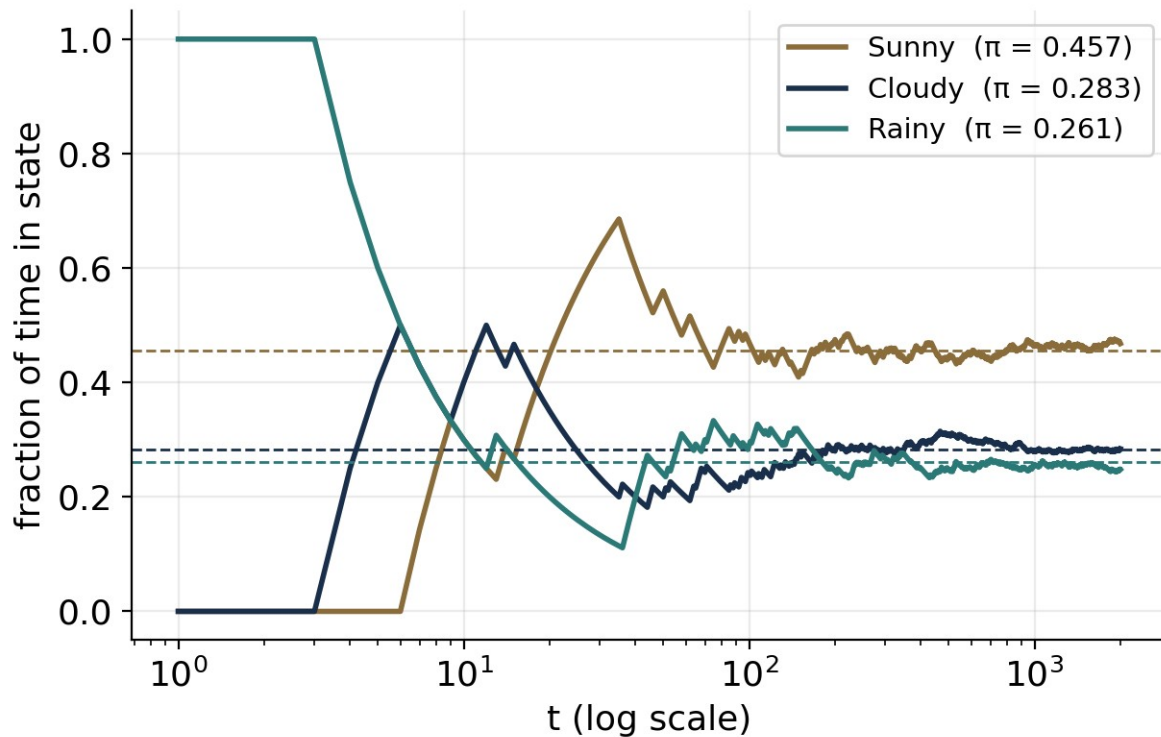


$$\pi = (0.457, 0.283, 0.261)$$

어떤 상태에서 출발해도 몇 단계 뒤에는 π 로 수렴 $\rightarrow$ 출발점을 " 잊는다 "

Ergodic theorem: 한 경로의 시간 평균 = 정상분포의 기댓값

$$\frac{1}{n} \sum_{t=1}^n h(X^{(t)}) \xrightarrow{a.s.} E_{\pi}[h(X)] \quad (\text{irreducible, aperiodic, positive recurrent})$$



- 조건 : **irreducible** (모든 상태로 갈 수 있음), **aperiodic** (주기가 없음), **positive recurrent**
- 한 경로 10 만 단계의 체류 비율 : 0.454, 0.283, 0.263 vs $\pi = 0.457, 0.283, 0.261$
- $X^{(t)}$ 들은 독립이 아니지만 **대수의 법칙이 성립**

MCMC의 발상 : 거꾸로 본다

P가 주어지고 π 를 구하는 대신, 목표분포 f 를 정상분포로 갖는 P를 만든다 → chain의 경로를 f 의 표본처럼 사용

Detailed balance: 정상분포를 보장하는 충분조건

$$\pi_i p_{ij} = \pi_j p_{ji} \quad \forall i, j \quad (\text{detailed balance, reversibility})$$

증명 : 양변을 i 에 대해 더하면

$$\sum_i \pi_i p_{ij} = \sum_i \pi_j p_{ji} = \pi_j \sum_i p_{ji} = \pi_j \implies \pi P = \pi$$

연속 상태공간 (MCMC 의 일반적인 경우) : 전이 kernel $p(x \rightarrow x^*)$ 에 대해 같은 논리

$$f(x) p(x \rightarrow x^*) = f(x^*) p(x^* \rightarrow x) \implies \int f(x) p(x \rightarrow x^*) dx = f(x^*)$$

의미 : $i \rightarrow j$ 로 흐르는 확률 질량 = $j \rightarrow i$ 로 흐르는 확률 질량 . 정상분포를 풀지 않고 " 쌍별 균형 " 만 확인하면 된다 . 다음 장의 Metropolis-Hastings 는 바로 이 조건을 만족하도록 설계된 전이 규칙이다 .

R 코드 : 정상분포와 ergodic 평균

```
## 3-state Markov chain: 1 = Sunny, 2 = Cloudy, 3 = Rainy
P <- matrix(c(0.7, 0.2, 0.1,
              0.3, 0.4, 0.3,
              0.2, 0.3, 0.5), 3, 3, byrow = TRUE)
## stationary distribution: left eigenvector of P with eigenvalue 1
e <- eigen(t(P))
pi.st <- Re(e$vectors[, 1]); pi.st <- pi.st / sum(pi.st)
round(pi.st, 4)
## pi_t = pi_0 P^t converges to pi.st from any start
p0 <- c(0, 0, 1)
for (t in 1:10) p0 <- p0 %*% P
round(p0, 4)
## ergodic theorem: long-run frequencies of ONE simulated path
set.seed(1)
n <- 100000; x <- numeric(n); x[1] <- 3
for (t in 2:n) x[t] <- sample(1:3, 1, prob = P[x[t - 1], ])
round(table(x) / n, 4)
```

포인트

- $\text{eigen}(t(P))$ 의 첫 고유벡터를 정규화
- $\pi_0 P^t$ 를 반복해서 곱해도 같은 값
- $\text{sample}(\dots, \text{prob} = P[\text{현재 상태}, \cdot])$ 로 경로 생성

출력 : $\pi = (0.4565, 0.2826, 0.2609)$, 한 경로의 체류 비율 = $(0.4543, 0.2829, 0.2627)$

02

Metropolis–Hastings 알고리즘

제안하고 , 비율 R 로 수락하거나 제자리에 머문다 (Metropolis et al. 1953; Hastings 1970)

알고리즘

- 1 현재 $x^{(t)}$ 에서 후보 $X^* \sim g(\cdot | x^{(t)})$ 를 제안
- 2 Metropolis-Hastings 비율 R 계산
- 3 확률 $\min\{1, R\}$ 로 이동, 아니면 $x^{(t+1)} = x^{(t)}$

장점

- f 는 상수배까지만 알면 됨 (비율에서 상쇄) $\rightarrow$ 베이지 사후분포
- envelope 상수 ($\sup f/g$) 가 필요 없음 (rejection sampling 과 비교)
- proposal g 는 거의 자유롭게 선택

$$R(x^{(t)}, X^*) = \frac{f(X^*) g(x^{(t)} | X^*)}{f(x^{(t)}) g(X^* | x^{(t)})}$$

$$X^{(t+1)} = \begin{cases} X^* & \text{with probability } \min\{1, R(x^{(t)}, X^*)\} \\ x^{(t)} & \text{otherwise} \end{cases}$$

주의

- 기각되어도 현재 값을 한 번 더 기록 (버리지 않음 !)
- 표본들은 독립이 아닌 상관된 sequence
- 초기값의 영향이 사라질 때까지 burn-in 필요

왜 맞는가 : MH kernel 은 detailed balance 를 만족

$x^* \neq x$ 로 이동하는 전이 밀도 = (제안될 확률) $\times$ (수락될 확률)

$$p(x \rightarrow x^*) = g(x^* | x) \min \left\{ 1, \frac{f(x^*) g(x | x^*)}{f(x) g(x^* | x)} \right\} \quad (x^* \neq x)$$

양변에 $f(x)$ 를 곱하면 x 와 x^* 에 대해 대칭인 식이 된다

$$f(x) p(x \rightarrow x^*) = \min \left\{ f(x) g(x^* | x), f(x^*) g(x | x^*) \right\} = f(x^*) p(x^* \rightarrow x)$$

→ detailed balance 성립 → f 가 chain 의 정상분포 . g 의 support 가 f 의 support 를 덮으면 (irreducible) ergodic theorem 에 의해 표본평균이 $E_f[h(X)]$ 로 수렴 .

proposal g 의 두 가지 대표적 선택 : ① independence chain $g(x^* | x) = g(x^*)$ ② random walk chain $x^* = x + \epsilon$

Independence chain

$$g(x^* | x) = g(x^*) \implies R = \frac{f(X^*)/g(X^*)}{f(x^{(t)})/g(x^{(t)})} = \frac{w(X^*)}{w(x^{(t)})}$$

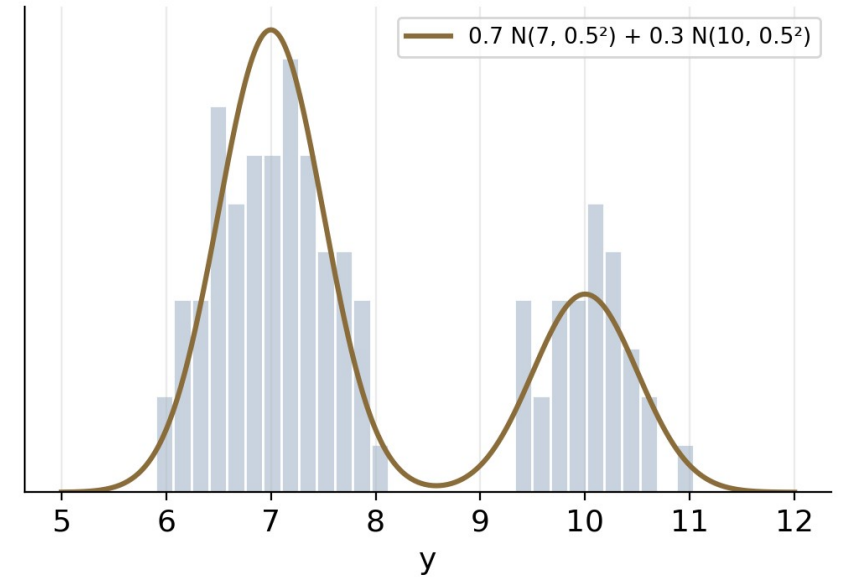
$w = f/g$ 는 importance weight 와 같음 : SIR 처럼 g 가 f 와 비슷하고 꼬리가 두꺼워야 잘 작동

예제 : 혼합비율 δ 의 사후분포 ($n = 100$, 참값 $\delta = 0.7$)

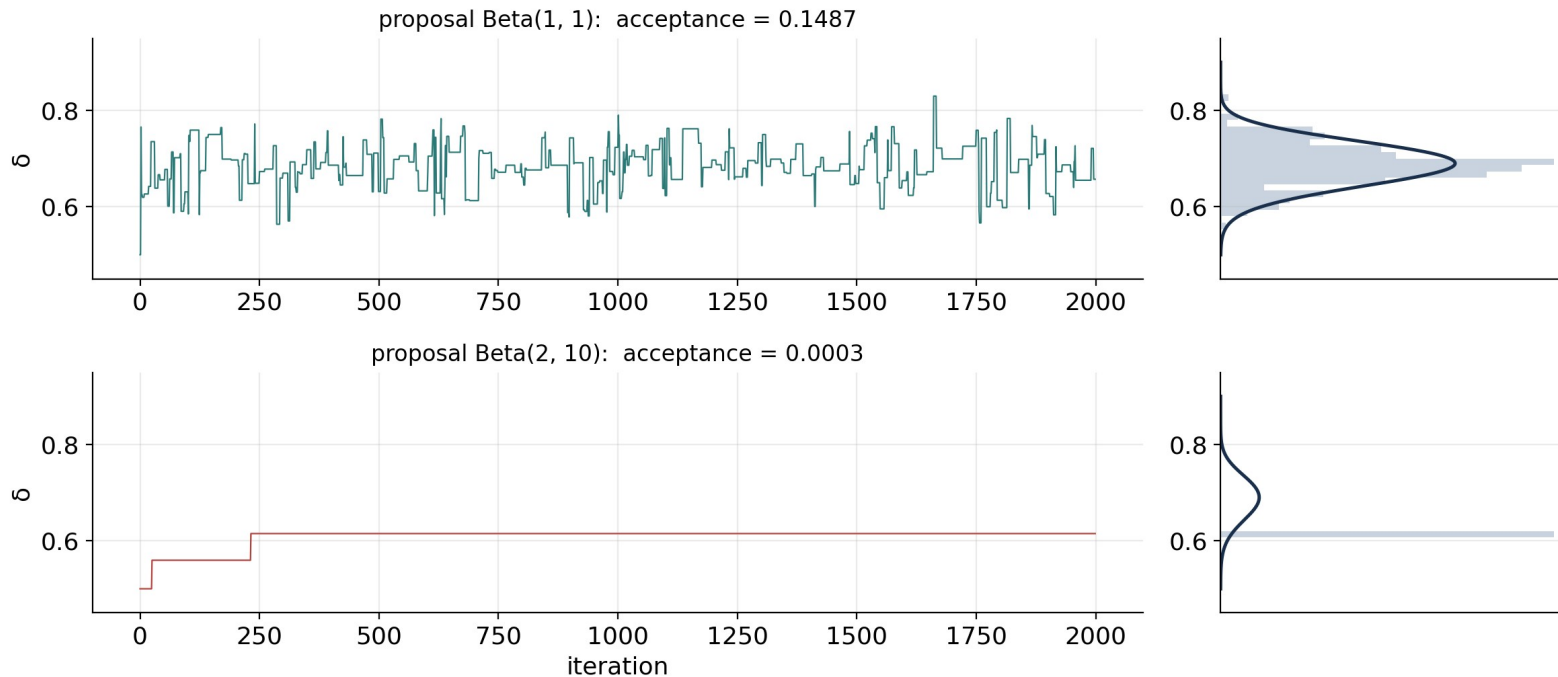
$$y_i \stackrel{iid}{\sim} \delta N(7, 0.5^2) + (1 - \delta) N(10, 0.5^2), \quad \delta \sim \text{Unif}(0, 1)$$

$$f(\delta | \mathbf{y}) \propto \prod_{i=1}^{100} \left[\delta \phi(y_i; 7, 0.5^2) + (1 - \delta) \phi(y_i; 10, 0.5^2) \right]$$

δ 의 사후분포는 표준분포가 아니고 정규화 상수도 모름 $\rightarrow$ MCMC



Independence chain: proposal 선택의 영향



- **Beta(1,1):** 수락률 0.149 로 낮지만 사후분포 전체를 탐색, 사후평균 0.686
- **Beta(2,10):** 평균 0.17 근처만 제안 → 10,000 번 중 3 번만 이동 . 한 값에 계속 붙어 있음
- 나쁜 chain 도 "수렴한 것처럼" 평평한 trace 를 보일 수 있음 → trace plot 을 반드시 확인

오른쪽 작은 그림 : burn-in 이후 표본의 히스토그램 vs 수치적으로 구한 참 사후밀도

R 코드 : Independence chain

```
## Data:  $y_i \sim \delta N(7, 0.5^2) + (1 - \delta) N(10, 0.5^2)$ ,  $n = 100$ ,  $\delta = 0.7$ 
set.seed(2)
y <- ifelse(runif(100) < 0.7, rnorm(100, 7, 0.5), rnorm(100, 10, 0.5))
## log posterior of delta with Unif(0,1) prior (up to a constant)
logpost <- function(d) sum(log(d * dnorm(y, 7, 0.5) + (1 - d) * dnorm(y, 10, 0.5)))

## Independence chain Metropolis-Hastings, proposal  $g = \text{Beta}(a, b)$ 
mh_indep <- function(N, a, b, x0 = 0.5) {
  x <- numeric(N); x[1] <- x0; acc <- 0
  for (t in 2:N) {
    xs <- rbeta(1, a, b) # candidate
    logR <- logpost(xs) - logpost(x[t - 1]) +
      dbeta(x[t - 1], a, b, log = TRUE) - dbeta(xs, a, b, log = TRUE)
    if (log(runif(1)) < logR) { x[t] <- xs; acc <- acc + 1 }
    else x[t] <- x[t - 1]
  }
  list(x = x, acc.rate = acc / (N - 1))
}
good <- mh_indep(10000, 1, 1) # Beta(1,1) = Unif(0,1)
bad <- mh_indep(10000, 2, 10) # Beta(2,10): mass near 0.17, far from target
c(good = good$acc.rate, bad = bad$acc.rate)
c(mean(good$x[-(1:500)]), mean(bad$x[-(1:500)]))
```

포인트

- log 스케일로 R 계산 (underflow 방지)
- dbeta 항이 g의 비율
- 기각 시에도 $x[t] \leftarrow x[t-1]$ 기록

출력 : 수락률 0.149 (Beta(1,1)) vs 0.0003 (Beta(2,10))

03

Random Walk Chain

현재 위치 주변에서 제안 : 대칭이면 비율이 f 만으로 결정된다

Random walk Metropolis

$$X^* = x^{(t)} + \epsilon, \quad \epsilon \sim h \text{ symmetric (e.g. } N(0, \sigma^2))$$

$$g(x^* | x) = g(x | x^*) \implies R = \frac{f(X^*)}{f(x^{(t)})} \quad (\text{Metropolis, 1953})$$

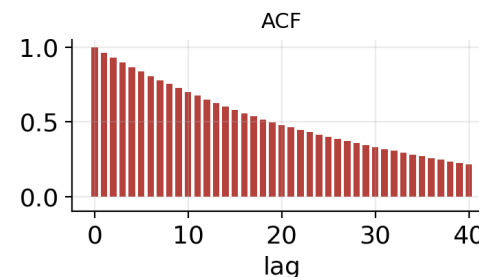
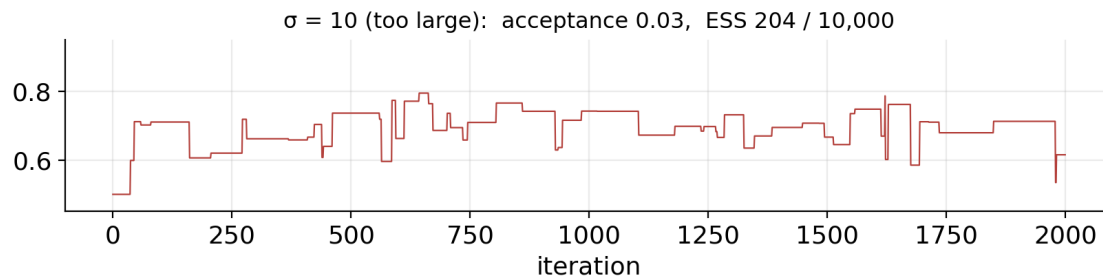
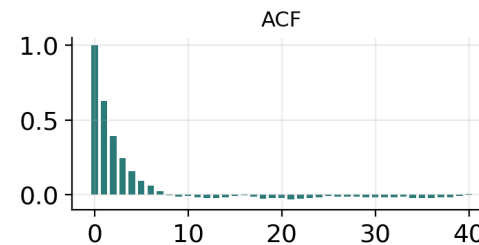
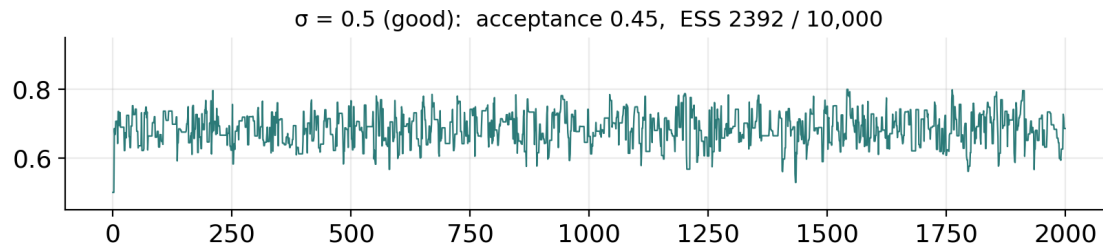
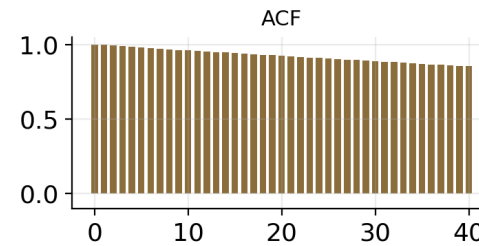
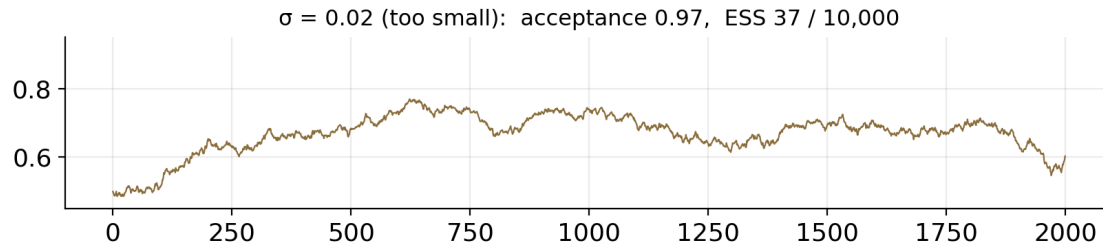
$f(X^*) \geq f(x)$ 이면 항상 이동, 낮은 곳으로는 비율만큼만 이동 $\rightarrow$ 높은 밀도 영역에 오래 머무름

Support 가 제한된 모수 : 변환 후 random walk (Jacobian 필요)

$$U = \text{logit}(\delta) = \log \frac{\delta}{1 - \delta}, \quad f_U(u) = f_\delta(\text{expit}(u)) \text{expit}(u)(1 - \text{expit}(u))$$

$\delta \in (0,1)$ 에서 그대로 걸으면 0, 1 밖으로 나가는 제안이 낭비됨. $U = \text{logit}(\delta)$ 로 바꾸면 실수 전체에서 걷고, 표본을 $\text{expit}(U)$ 로 되돌리면 δ 의 표본이 된다.

Proposal scale σ 의 선택 : 너무 작아도 , 너무 커도 나쁘다



- **작은 σ :** 거의 다 수락 (0.97) 되지만 조금씩만 움직임 $\rightarrow$ 강한 자기상관
- **큰 σ :** 대부분 기각 (0.03) $\rightarrow$ 같은 값 반복
- **적당한 σ :** 수락률 0.45, ESS 가 가장 큼
- 경험칙 : 1 차원 약 0.44, 고차원 약 0.23 (Roberts, Gelman & Gilks, 1997)

사후평균 (burn-in 500 제외): 0.680 / 0.686 / 0.692 — 세 chain 모두 같은 분포를 목표로 하지만 효율이 다름

R 코드 : Random walk chain

```
## Random walk chain on u = logit(delta) (same data / posterior as mh_indep.R)
set.seed(2)
y <- ifelse(runif(100) < 0.7, rnorm(100, 7, 0.5), rnorm(100, 10, 0.5))
logpost <- function(d) sum(log(d * dnorm(y, 7, 0.5) + (1 - d) * dnorm(y, 10, 0.5)))
## target in u: f_U(u) = f_delta(expit(u)) * |d delta / du| (Jacobian)
logpost_u <- function(u) { d <- plogis(u); logpost(d) + log(d * (1 - d)) }

rw_chain <- function(N, s, u0 = 0) {
  u <- numeric(N); u[1] <- u0; acc <- 0
  for (t in 2:N) {
    us <- u[t - 1] + rnorm(1, 0, s) # symmetric proposal
    if (log(runif(1)) < logpost_u(us) - logpost_u(u[t - 1])) { # Metropolis ratio
      u[t] <- us; acc <- acc + 1
    } else u[t] <- u[t - 1]
  }
  list(delta = plogis(u), acc.rate = acc / (N - 1))
}
set.seed(3)
small <- rw_chain(10000, 0.02); mid <- rw_chain(10000, 0.5); big <- rw_chain(10000, 10)
c(small = small$acc.rate, mid = mid$acc.rate, big = big$acc.rate)
```

포인트

- logpost_u 에 log Jacobian $\log \delta(1-\delta)$ 를 더함
- 대칭 proposal 이므로 g 항이 없음
- s 를 바꿔 수락을 조정

출력 : 수락률 $\sigma = 0.02$: 0.97, $\sigma = 0.5$: 0.45, $\sigma = 10$: 0.03

04

Gibbs Sampling

다변량 분포를 한 좌표씩, 조건부 분포에서 차례로 뽑는다 (Geman & Geman, 1984)

알고리즘

$$X_j^{(t+1)} \sim f(x_j \mid x_1^{(t+1)}, \dots, x_{j-1}^{(t+1)}, x_{j+1}^{(t)}, \dots, x_p^{(t)}), \quad j = 1, \dots, p$$

한 번의 반복 = 모든 좌표를 한 바퀴 갱신 . 방금 갱신한 값을 바로 다음 좌표의 조건에 사용한다 .

$$g(\mathbf{x}^* \mid \mathbf{x}) = f(x_j^* \mid \mathbf{x}_{-j}) \implies R = 1 \quad (\text{always accept})$$

언제 쓰나

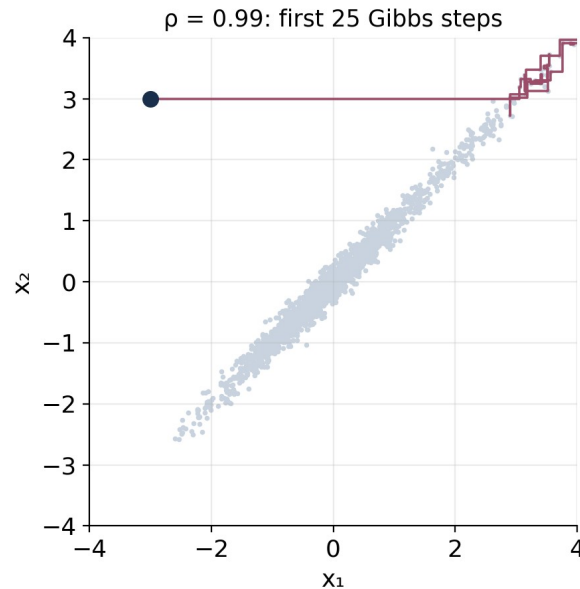
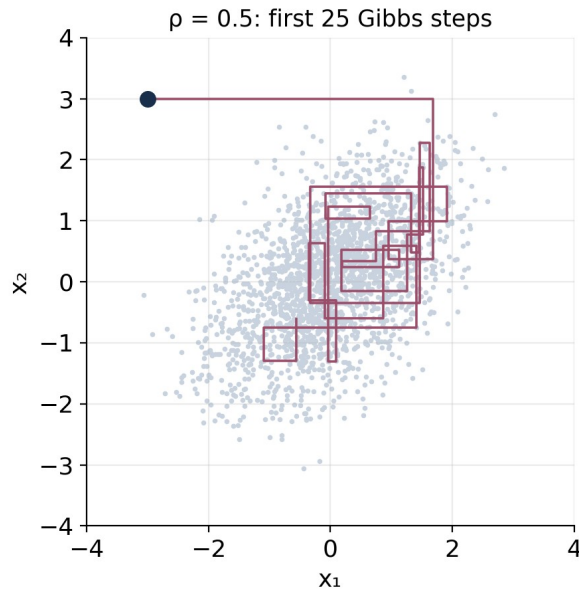
- full conditional $f(x_j \mid \text{나머지})$ 에서 직접 뽑을 수 있을 때
- conjugate prior 를 쓴 계층모형 (Bayesian hierarchical model)
- 결측 · 잠재변수를 모수와 번갈아 뽑는 data augmentation

변형

- Metropolis-within-Gibbs: 뽑기 어려운 좌표만 MH 한 단계로
- Blocking: 상관이 큰 좌표들을 묶어서 함께 갱신
- Random scan: 갱신할 좌표를 무작위로 선택

예제 1: 이변량 정규분포 — 상관이 크면 느리다

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim N_2\left(\mathbf{0}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right) : \quad X_1 \mid x_2 \sim N(\rho x_2, 1 - \rho^2), \quad X_2 \mid x_1 \sim N(\rho x_1, 1 - \rho^2)$$



- 각 단계는 좌표축에 평행하게만 움직임 (계단 모양)
- $\rho = 0.5$: lag-1 자기상관 0.25, 금방 분포 전체를 돌아다님
- $\rho = 0.99$: 조건부 분산 $1 - \rho^2 = 0.02 \rightarrow$ 한 걸음이 아주 작음, lag-1 자기상관 0.98
- 해결책 : 두 좌표를 함께 뽑는 blocking, 또는 모수 재설정 (reparameterization)

예제 2: 정규모형의 사후분포 (semi-conjugate prior)

$$y_i \mid \mu, \sigma^2 \stackrel{iid}{\sim} N(\mu, \sigma^2), \quad \mu \sim N(\mu_0, \tau_0^2), \quad \sigma^2 \sim \text{InvGamma}(a, b)$$

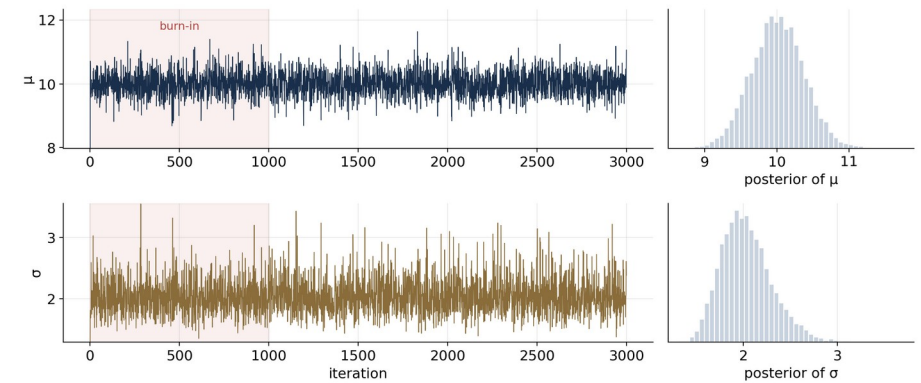
Full conditional 은 둘 다 표준분포 → Gibbs 로 번갈아 뽑음

$$\mu \mid \sigma^2, \mathbf{y} \sim N\left(\frac{\mu_0/\tau_0^2 + n\bar{y}/\sigma^2}{1/\tau_0^2 + n/\sigma^2}, \frac{1}{1/\tau_0^2 + n/\sigma^2}\right)$$

$$\sigma^2 \mid \mu, \mathbf{y} \sim \text{InvGamma}\left(a + \frac{n}{2}, b + \frac{1}{2} \sum_{i=1}^n (y_i - \mu)^2\right)$$

n = 30, 10,000 회 (burn-in 1,000)	사후평균	95% 신용구간
μ	10.007	(9.28, 10.73)
σ	2.036	(1.57, 2.66)

데이터 : $\bar{y} = 10.023, s = 1.983$ (참값 $\mu = 10, \sigma = 2$). prior: $\mu_0 = 0, \tau_0^2 = 100, a = b = 0.01$



R 코드 (1): 이변량 정규 Gibbs

```
## Gibbs sampler for (X1, X2) ~ N2(0, [[1, rho], [rho, 1]])
## full conditionals: X1 | X2 ~ N(rho X2, 1 - rho^2), X2 | X1 ~ N(rho X1, 1 - rho^2)
gibbs_bvn <- function(N, rho, x0 = c(-3, 3)) {
  x <- matrix(NA, N, 2); x[1, ] <- x0
  for (t in 2:N) {
    x[t, 1] <- rnorm(1, rho * x[t - 1, 2], sqrt(1 - rho^2))
    x[t, 2] <- rnorm(1, rho * x[t, 1], sqrt(1 - rho^2)) # uses NEW x1
  }
  x
}
set.seed(4)
g5 <- gibbs_bvn(5000, 0.5)
g99 <- gibbs_bvn(5000, 0.99)
c(cor(g5)[1, 2], cor(g99)[1, 2])
c(acf(g5[, 1], plot = FALSE)$acf[2], acf(g99[, 1], plot = FALSE)$acf[2])
```

포인트

- x_2 를 뽑을 때 방금 뽑은 $x[t, 1]$ 을 사용
- ρ 를 0.5 $\rightarrow$ 0.99 로 바꿔 ACF 비교

출력 : lag-1 자기상관 $\rho = 0.5$: 0.252, $\rho = 0.99$: 0.976

R 코드 (2): 정규모형 Gibbs

```
## y_i ~ N(mu, sigma2); priors mu ~ N(mu0, tau0^2), sigma2 ~ InvGamma(a, b)
set.seed(5)
y <- rnorm(30, mean = 10, sd = 2); n <- length(y); ybar <- mean(y)
mu0 <- 0; tau02 <- 100; a <- 0.01; b <- 0.01
N <- 10000; mu <- numeric(N); s2 <- numeric(N); mu[1] <- 0; s2[1] <- 1
for (t in 2:N) {
  ## mu | sigma2, y ~ Normal
  prec <- 1 / tau02 + n / s2[t - 1]
  mu[t] <- rnorm(1, (mu0 / tau02 + n * ybar / s2[t - 1]) / prec, sqrt(1 / prec))
  ## sigma2 | mu, y ~ InvGamma(a + n/2, b + sum((y - mu)^2)/2)
  s2[t] <- 1 / rgamma(1, a + n / 2, b + sum((y - mu[t])^2) / 2)
}
keep <- 1001:N # burn-in 1000
c(mu = mean(mu[keep]), sigma = mean(sqrt(s2[keep])))
quantile(mu[keep], c(0.025, 0.975)); c(ybar = ybar, sd = sd(y))
```

포인트

- InvGamma 는 1/rgamma 로 생성
- prec = 사후 정밀도 (분산의 역수)
- 처음 1,000 회는 burn-in 으로 버림

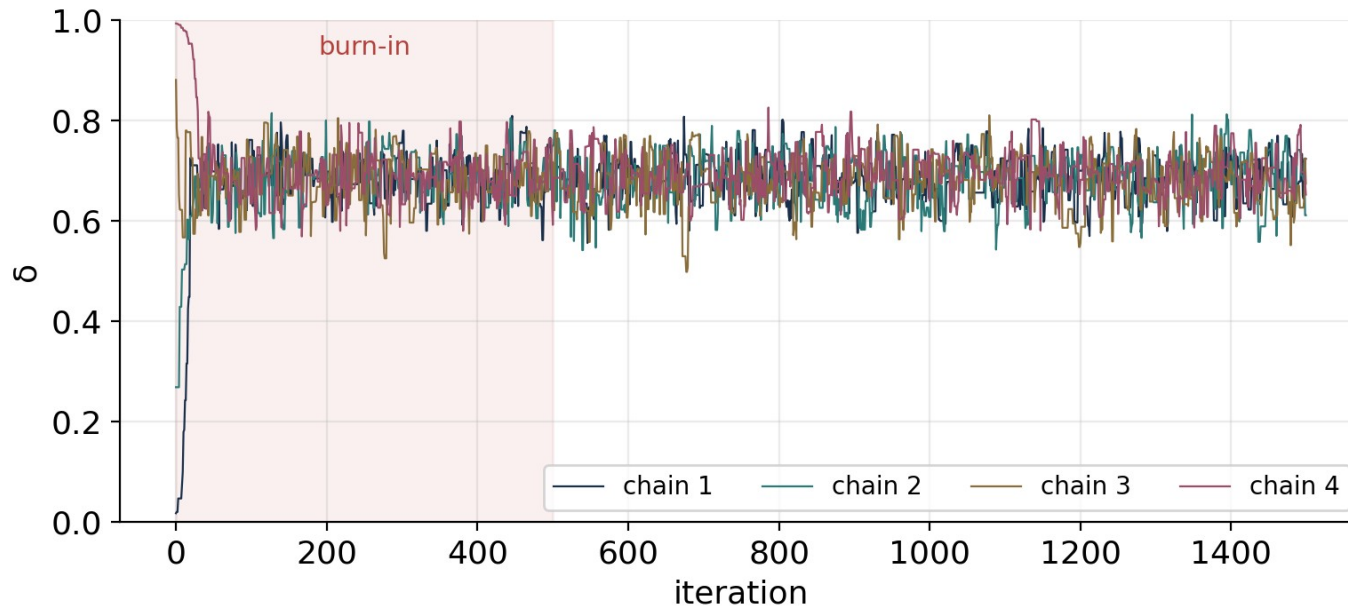
출력 : $E[\mu | y] = 10.007$, $E[\sigma | y] = 2.036$

05

수렴 진단과 구현

MCMC 결과를 믿기 전에 반드시 확인해야 할 것들

여러 chain, burn-in, 그리고 Gelman–Rubin $\hat{R}$



진단 도구

- **Trace plot:** chain 들이 같은 영역에서 섞이는가
- **Burn-in:** 초기값의 영향이 남은 앞부분을 버림
- **$\hat{R}$:** chain 간 분산 vs chain 내 분산 . 처음 100 회 1.149 → burn-in 이후 1.000 (1.1 미만이면 양호)
- **ACF 와 ESS:** 다음 슬라이드

$$\hat{R} = \sqrt{\frac{\frac{n-1}{n}W + \frac{1}{n}B}{W}} \xrightarrow{\text{converged}} 1 \quad (W : \text{within}, B : \text{between chains})$$

서로 멀리 떨어진 초기값 (logit 스케일 -4, -1, 2, 5) 에서 출발한 4 개의 random walk chain

자기상관과 유효표본크기 (ESS)

$$\text{ESS} = \frac{n}{1 + 2 \sum_{k \geq 1} \rho_k} \quad (\rho_k : \text{lag-}k \text{ autocorrelation})$$

Random walk chain (n = 10,000)	수락률	ESS	ESS / n
$\sigma = 0.02$	0.97	37	0.004
$\sigma = 0.5$	0.45	2,392	0.239
$\sigma = 10$	0.03	204	0.020

- MCMC 표본 10,000 개 $\neq$ 독립 표본 10,000 개
- MC 표준오차는 $\sqrt{\text{Var}/\text{ESS}}$ 로 계산해야 함
- Thinning (k 개마다 하나): 저장 공간 절약용이지 정확도를 높이지는 않음

실무 체크리스트

- 초기값을 다르게 한 chain 3-4 개
- trace plot 과 ACF 확인
- $\hat{R} < 1.1$, 충분한 ESS (예: ≥ 400)
- proposal scale 조정 (수락률 0.2-0.5)
- R 패키지 coda: effectiveSize(), gelman.diag()

R 코드 : ESS 와 $\hat{R}$

```
## Convergence diagnostics for the random walk chain (rw.R must be run first)
ess <- function(x) {
  # effective sample size
  r <- acf(x, lag.max = 200, plot = FALSE)$acf[-1]
  k <- which(r < 0.05)[1]; if (is.na(k)) k <- length(r)
  length(x) / (1 + 2 * sum(r[1:k]))
}
rhat <- function(chains) {
  # Gelman-Rubin (columns = chains)
  n <- nrow(chains); m <- ncol(chains)
  B <- n * var(colMeans(chains)); W <- mean(apply(chains, 2, var))
  sqrt(((n - 1) / n * W + B / n) / W)
}
set.seed(6)
starts <- c(-4, -1, 2, 5)
ch <- sapply(starts, function(u0) rw_chain(5000, 0.5, u0)$delta)
burn <- 1:500
c(Rhat = rhat(ch[-burn, ]), ESS.small = ess(small$delta), ESS.mid = ess(mid$delta),
  ESS.big = ess(big$delta))
```

포인트

- rw.R 의 rw_chain() 을 먼저 실행
- ess(): ACF 를 lag 합이 작아질 때까지 더함 (단순 버전)
- rhat(): 열 = chain

출력 : $\hat{R} = 1.0002$, ESS = 37 / 2,392 / 204

정리

	Independence chain	Random walk chain	Gibbs sampling
제안	$g(x^*)$ (현재 값과 무관)	$x^* = x + \varepsilon$ (대칭)	full conditional 에서 한 좌표씩
수락 비율 R	$w(x^*) / w(x)$, $w = f/g$	$f(x^*) / f(x)$	항상 1
잘 되는 경우	g 가 f 와 비슷하고 꼬리가 두꺼울 때	거의 항상 사용 가능 (σ 조정 필요)	조건부 분포에서 쉽게 뽑을 때
약점	g 가 나쁘면 한 값에 고착	고차원 · 상관 큰 경우 느림	좌표 간 상관이 크면 느림

공통 원리 : detailed balance 로 f 를 정상분포로 만들고 , ergodic theorem 으로 경로 평균을 사용 . 결과를 쓰기 전에 수렴 진단 !

References

Givens & Hoeting (2013). Computational Statistics, 2nd ed., Ch. 1.7, 7, 8. · Robert & Casella (2004). Monte Carlo Statistical Methods, Ch. 6–10.
 Metropolis et al. (1953), J. Chem. Phys. · Hastings (1970), Biometrika · Geman & Geman (1984), IEEE PAMI · Gelfand & Smith (1990), JASA
 Gelman & Rubin (1992), Statist. Sci. · Roberts, Gelman & Gilks (1997), Ann. Appl. Probab.